Enumerators

As powerful and flexible as the Ruby enumeration methods are, the ones we've seen all have the same structure. A block is passed to an object, and that object controls how it interacts with the block and traverses the collection.

As useful as that structure is, it still doesn't cover all the cases where iteration is useful. Sometimes, you want an external object or method to control how the collection is traversed. You may need a more complicated kind of access to the block methods. For example, you might want to iterate over two collections in parallel, which is very difficult using Ruby's internal iterator scheme.

Fortunately, Ruby comes with a built-in Enumerator class, which implements external iterators in Ruby for just such occasions. An *external* iterator is an iterator where you control the iteration behavior outside the iterator itself, meaning we have a specific way of explicitly triggering when the iterator should move to the next element in its collection.

The Enumerator class is not to be confused with the Enumerable module, which we will discuss in Chapter 6, Sharing Functionality: Inheritance, Modules, and Mixins, on page 101. The Enumerable module is a mixin that provides functionality to a variety of classes. The Enumerator class is a class that allows for external iterators.

You can create an Enumerator object by calling the to_enum method (or its synonym, enum_for) on a collection such as an array or a hash. Once you have an Enumerator, you can access the next element in the collection with the method next:

```
a = [1, 3, "cat"]
enum_a = a.to_enum
enum_a.next # => 1
enum_a.next # => 3

h = {dog: "canine", fox: "vulpine"}
enum_h = h.to_enum
enum_h.next # => [:dog, "canine"]
enum_h.next # => [:fox, "vulpine"]
```

By default, the new enumerator uses the each method as the way it walks the underlying enumeration, but you can use any method that successively yields values to a block:

```
a = [1, 3, "cat"]
enum_a = a.to_enum(:reverse_each)
enum_a.next # => "cat"
enum_a.next # => 3
```

Most of Ruby's internal iterator methods—the ones that normally yield successive values to a block—will return an Enumerator object if called without a block:

```
a = [1, 3, "cat"]

enum_a = a.each

enum_a.next # => 1
enum_a.next # => 3
```

Ruby has a method called loop that does nothing but repeatedly invoke its block. Typically, your code in the block will look for an ending condition and break out of the loop when that